# Red social de recetas (FoodLoop) Documento de Especificaciones, Requerimientos y Criterios de Aceptación de Software (DERCAS)

Equipo de Desarrollo de Sistemas — FoodLoop

Departamento de Desarrollo de Sistemas

| | |
|---|---|
| **Proyecto** | FoodLoop: red social de recetas culinarias para compartir, valorar y seguir contenido gastronómico. |
| **Versión del documento** | 1.0 |
| **Fecha** | Octubre de 2026 |
| **Estado** | En desarrollo (API REST Express + MongoDB, frontend Angular y almacenamiento GridFS) |
| **Elaborado por** | Equipo de Desarrollo de Sistemas — FoodLoop |
| **Fuente de verdad** | Documentación del proyecto (docs/) y código fuente |

## Historial de revisiones

| Fecha | Versión | Descripción |
|---|---|---|
| 15 de octubre de 2026 | 1.0 | Primera versión del DERCAS adaptado a la estructura APA7 del modelo de referencia. |

# Índice

**1. Definiciones, acrónimos y abreviaturas** **5**

**2. Descripción del sistema solicitado** **7**

**3. Definición de actores** **8**

**4. Procesos a sistematizar** **9**
4.1. P01: Autenticación y sesión . . . . . 9
4.2. P02: Gestión de recetas y visibilidad . . . . . 9
4.3. P03: Gestión de imágenes . . . . . 10
4.4. P04: Interacciones sociales . . . . . 11
4.5. P05: Comentarios y reseñas . . . . . 11
4.6. P06: Seguimiento, feed y notificaciones . . . . . 12

**5. Diagrama de flujo de los procesos** **13**
5.1. Autenticación y sesión . . . . . 14
5.2. Recetas y visibilidad . . . . . 15
5.3. Imágenes . . . . . 16
5.4. Interacciones sociales . . . . . 17
5.5. Comentarios y reseñas . . . . . 17
5.6. Seguimiento, feed y notificaciones . . . . . 18
5.7. Visión general del sistema . . . . . 19

**6. Casos de uso** **20**
6.1. Casos de uso del módulo de autenticación . . . . . 20
6.1.1. Caso de uso “CU-01: Registrarse” . . . . . 20
6.1.2. Caso de uso “CU-02: Iniciar sesión” . . . . . 21
6.1.3. Caso de uso “CU-03: Cerrar sesión” . . . . . 22
6.2. Casos de uso del módulo de recetas . . . . . 24
6.2.1. Caso de uso “CU-04: Crear receta” . . . . . 24
6.2.2. Caso de uso “CU-05: Listar y ver recetas” . . . . . 25
6.2.3. Caso de uso “CU-06: Editar, eliminar y cambiar visibilidad” . . . . . 26
6.3. Casos de uso del módulo de imágenes . . . . . 28
6.3.1. Caso de uso “CU-07: Subir imagen” . . . . . 28
6.3.2. Caso de uso “CU-08: Reemplazar o quitar imagen” . . . . . 29
6.3.3. Caso de uso “CU-09: Ver imagen” . . . . . 30
6.4. Casos de uso del módulo de interacciones sociales . . . . . 31
6.4.1. Caso de uso “CU-10: Me gusta” . . . . . 31
6.4.2. Caso de uso “CU-10b: Guardar receta” . . . . . 32
6.4.3. Caso de uso “CU-10c: Compartir receta” . . . . . 33
6.4.4. Caso de uso “CU-10d: Reportar receta” . . . . . 34
6.5. Casos de uso del módulo de comentarios y reseñas . . . . . 35
6.5.1. Caso de uso “CU-11: Leer comentarios y reseñas” . . . . . 35
6.5.2. Caso de uso “CU-12: Comentar y responder” . . . . . 36
6.5.3. Caso de uso “CU-13: Reseñar” . . . . . 37
6.5.4. Caso de uso “CU-14: Editar o borrar contenido propio” . . . . . 38
6.6. Casos de uso del módulo de seguimiento, feed y notificaciones . . . . . 39
6.6.1. Caso de uso “CU-14b: Seguir o dejar de seguir” . . . . . 39
6.6.2. Caso de uso “CU-15: Ver seguidores, seguidos y feed” . . . . . 40

6.6.3. Caso de uso “CU-16: Consultar la campana de notificaciones” . . . . . 41

**7. Diagrama de secuencia** **42**
7.1. Secuencia de registro e inicio de sesión . . . . . 43
7.2. Secuencia de creación de receta y gestión de imagen . . . . . 44
7.3. Secuencia de comentar y reseñar . . . . . 45
7.4. Secuencia de seguir usuarios y consultar el feed . . . . . 46

**8. Diagramas de estado** **46**
8.1. Estados de la receta . . . . . 47
8.2. Estados de la imagen . . . . . 48
8.3. Estados de la campana de notificaciones . . . . . 49

**9. Diagrama ER** **49**

**10. Requerimientos no funcionales** **50**
10.1. Arquitectura de tres capas . . . . . 50
10.2. Autenticación y autorización . . . . . 50
10.3. Seguridad de datos . . . . . 51
10.4. Almacenamiento . . . . . 52
10.5. Integración . . . . . 52

**11. Planificación de ciclos de desarrollo** **53**

**12. Criterios de aceptación** **53**
12.1. Autenticación y gestión de usuarios . . . . . 53
12.2. Recetas e imágenes . . . . . 54
12.3. Interacciones, comentarios y reseñas . . . . . 54
12.4. Seguimiento, feed y notificaciones . . . . . 54
12.5. Seguridad y calidad . . . . . 54

**13. Firmas de aceptación** **55**

## 1. Definiciones, acrónimos y abreviaturas

| **Término** | **Concepto** |
|---|---|
| API | Interfaz de programación de aplicaciones. Pieza de software que permite que dos aplicaciones se comuniquen entre sí; en FoodLoop es la capa REST en Node.js/Express que atiende al cliente Angular. |
| Autenticación | Proceso mediante el cual un usuario acredita su identidad con un correo y una contraseña, y recibe un token que lo identifica en las peticiones siguientes. |
| Backend | Arquitectura interna de la aplicación, no visible para el usuario. Concentra la lógica de negocio, valida las peticiones y responde con datos procesados. En FoodLoop lo compone la API REST en Node.js/Express con Mongoose. |
| Comentario | Texto asociado a una receta, con posibilidad de una respuesta de primer nivel. Su borrado es lógico. |
| CORS | *Cross-Origin Resource Sharing*. Mecanismo que permite que un navegador acepte peticiones desde orígenes distintos al que sirve la aplicación. |
| CRUD | Siglas de *Crear, Leer, Actualizar y Eliminar*; las cuatro operaciones básicas sobre los datos. |
| DERCAS | *Documento de Especificaciones, Requerimientos y Criterios de Aceptación de Software*. Documento que describe el sistema solicitado, sus actores, procesos, casos de uso, requisitos y criterios de aceptación. |
| `CastError` | Error de Mongoose al convertir un identificador con formato inválido en un `ObjectId`. |
| Feed | Lista paginada de actividad propia y de los usuarios seguidos por un usuario. |

| **Término** | **Concepto** |
|---|---|
| GridFS | Sistema de archivos de MongoDB que almacena archivos binarios dividiéndolos en fragmentos. FoodLoop guarda las imágenes en el bucket `uploads`. |
| Imagen | Archivo binario JPEG, PNG, WebP o GIF guardado en GridFS. Cada receta conserva el identificador de su única imagen vigente. |
| Interacción social | Acción alternable sobre una receta: me gusta, guardar, compartir. El reporte, en cambio, escribe un motivo y marca la receta. |
| JWT | *JSON Web Token*. Token firmado que identifica la sesión; en FoodLoop tiene una vigencia de 30 días y viaja en la cabecera `Authorization: Bearer`. |
| MIME | *Multipurpose Internet Mail Extensions*. Tipo declarado de un archivo, por ejemplo `image/jpeg`. |
| `ObjectId` | Identificador único de un documento de MongoDB. |
| Propietario | Usuario que creó una receta. Es el único que puede editarla, eliminarla, cambiar su visibilidad y gestionar su imagen. La propiedad se determina por receta. |
| Receta | Documento creado por un usuario con título, descripción, ingredientes, instrucciones, categoría, etiquetas y visibilidad. Puede tener una imagen en GridFS. |
| Reseña | Calificación de 1 a 5 con texto opcional. Un usuario puede dejar una sola reseña por receta; el promedio se recalcula con cada alta. |
| REST | Estilo de arquitectura de servicios sobre HTTP en el que los recursos se manipulan con los métodos del protocolo. |
| RNF | Requerimiento no funcional. Condición de calidad, seguridad o desempeño del sistema. |
| Seguimiento | Relación dirigida entre dos usuarios: uno sigue al otro. Es alternable y no admite el auto-seguimiento. |

| **Término** | **Concepto** |
|---|---|
| Saneado | Limpieza de las entradas recibidas del cliente (`sanitizeFields`) para evitar código o marcado no deseado antes de persistirlas. |
| SPA | *Single Page Application*. Aplicación web que ejecuta su contenido en una sola página, sin recargas completas. FoodLoop la implementa con Angular. |
| Token | Credencial de sesión que el cliente envía al servidor para acceder a las rutas protegidas. |
| Visibilidad | Atributo `publica` de una receta. Si es verdadero, cualquier usuario autenticado puede verla; si es falso, únicamente su propietario. |

## 2. Descripción del sistema solicitado

El proyecto parte de un recetario personal y evoluciona hacia una red social de recetas. La necesidad original era contar con un espacio propio para registrar recetas; el sistema solicitado amplía ese alcance con funciones sociales que permiten descubrir, valorar y seguir el contenido de otros usuarios.

Se solicita el desarrollo e implementación de una red social de recetas, denominada **FoodLoop**, que permita:

- El registro y la autenticación de usuarios mediante correo electrónico y contraseña.
- La creación, consulta, edición, eliminación y cambio de visibilidad pública o privada de recetas.
- El almacenamiento de una imagen por receta en el bucket GridFS `uploads`.
- Las interacciones sociales sobre cada receta: me gusta, guardar, compartir y reportar.
- Los comentarios, las respuestas de primer nivel y las reseñas con promedio de calificación.
- El seguimiento entre usuarios, el feed de actividad y una campana de notificaciones.

El sistema se construye sobre una arquitectura de tres capas: un cliente de página única desarrollado con Angular y TypeScript, una API REST implementada con Node.js, Express y Mongoose bajo el patrón Modelo–Vista–Controlador, y una base de datos MongoDB en la nube con GridFS para los archivos binarios. El desarrollo conserva y documenta los defectos conocidos en los capítulos de procesos, casos de uso y planificación.

## 3. Definición de actores

| **Nombre del actor** | **Descripción** |
|---|---|
| Visitante | Persona sin sesión iniciada. Puede registrarse, iniciar sesión y, si conoce la URL, ver una imagen pública. No puede llamar a rutas protegidas. |
| Usuario registrado | Persona autenticada. Puede crear y gestionar sus recetas, interactuar socialmente, comentar, reseñar, seguir a otros usuarios y consultar el feed. |
| Propietario de receta | Usuario registrado con derechos adicionales sobre una receta concreta: editarla, eliminarla, gestionar su imagen y cambiar su visibilidad. La propiedad se determina por receta, no por un rol global. |
| API REST (actor interno) | Componente de servidor que atiende las peticiones, verifica el token, valida la propiedad del recurso y ejecuta las operaciones de escritura y lectura sobre la base de datos. |
| Sistema de almacenamiento (actor externo) | MongoDB Atlas y GridFS, responsables de la persistencia de documentos e imágenes. |

No existe una cuenta administradora. La distinción entre usuario común y propietario depende de la propiedad de cada receta.

# 4. Procesos a sistematizar

A continuación se describen los procesos que el sistema debe automatizar. Cada proceso indica su nombre, descripción, los actores involucrados, sus entradas y salidas, las especificaciones de funcionamiento y las restricciones que condicionan su implementación. Los diagramas de flujo, los casos de uso y las secuencias se presentan en las secciones siguientes.

## 4.1. P01: Autenticación y sesión

| | |
|---|---|
| **Nombre** | P01: Autenticación y sesión |
| **Descripción** | Registro, inicio de sesión y cierre local de sesión. |
| **Actores** | Visitante y usuario registrado. |
| **Entradas** | Nombre, correo y contraseña. |
| **Salidas** | Identificador, nombre, correo y token JWT. |
| **Especificaciones de funcionamiento** | `POST /api/auth/register` crea el usuario con hash bcrypt; `POST /api/auth/login` compara el hash y emite un token de 30 días. El cierre de sesión elimina el token del navegador. El correo debe ser único y la contraseña debe tener al menos 8 caracteres. |
| **Restricciones adicionales** | Límite de 10 intentos cada 15 minutos y saneado del nombre y del correo en el registro. Respuestas `201`, `200`, `400`, `429` y `500`. |

## 4.2. P02: Gestión de recetas y visibilidad

| | |
|---|---|
| **Nombre** | P02: Gestión de recetas y visibilidad |
| **Descripción** | Creación, consulta, edición, eliminación y cambio de visibilidad de recetas. |
| **Actores** | Usuario registrado y propietario. |

| | |
|---|---|
| **Entradas** | Título, descripción, ingredientes, instrucciones, categoría y visibilidad. |
| **Salidas** | Receta creada, lista filtrada, detalle permitido o confirmación de borrado. |
| **Especificaciones de funcionamiento** | `POST /api/recipes`, `GET /api/recipes`, `GET /api/recipes?scope=public`, `GET /api/recipes/:id`, `PUT /api/recipes/:id` y `DELETE /api/recipes/:id`. El servidor asigna el propietario y acepta únicamente 11 campos editables. Las recetas nacen privadas; el detalle permite recetas propias o públicas; la edición y el borrado exigen propiedad. |
| **Restricciones adicionales** | Lista de campos editables, validadores Mongoose, límite de creación y saneado parcial. No hay búsqueda ni paginación en el servidor. Respuestas `201`, `200`, `400`, `404`, `429` y `500`. |

### 4.3. P03: Gestión de imágenes

| | |
|---|---|
| **Nombre** | P03: Gestión de imágenes |
| **Descripción** | Selección, optimización, subida, reemplazo, eliminación y visualización de imágenes. |
| **Actores** | Propietario para la gestión; cualquier persona con la URL para la visualización. |
| **Entradas** | Archivo de imagen e identificador de receta. |
| **Salidas** | Identificador GridFS asociado o receta sin imagen. |
| **Especificaciones de funcionamiento** | `POST /api/recipes/:id/image` con `multipart/form-data`; `PUT /api/recipes/:id` con imagen vacía; `GET /api/recipes/image/:fileId` público. Solamente se conserva la imagen vigente; la anterior se borra al reemplazar, quitar o eliminar la receta. |

| | |
|---|---|
| **Restricciones adicionales** | MIME de imagen, 25 MB en el navegador, 5 MB en el servidor y propiedad de la receta. No se verifican los bytes reales del archivo. Respuestas `200`, `400`, `404` y `500`. |

### 4.4. P04: Interacciones sociales

| | |
|---|---|
| **Nombre** | P04: Interacciones sociales |
| **Descripción** | Alternar me gusta, guardar, compartir y reportar recetas. |
| **Actores** | Usuario registrado. |
| **Entradas** | Identificador de receta y, para reportes, motivo. |
| **Salidas** | Estado alternado y contadores actualizados. |
| **Especificaciones de funcionamiento** | `POST /api/recipes/:id/like`, `/save`, `/share` y `/report`. El me gusta crea o elimina una fila en `likes`; guardar usa `saves`. Las acciones son alternables; dar me gusta a la receta propia responde `400`; compartir registra actividad. |
| **Restricciones adicionales** | Límite social de 30 acciones por minuto, límite de creación para reportes y saneado del motivo. Respuestas `200`, `201` (reporte), `400` (me gusta propio o reporte duplicado), `404`, `429` y `500`. |

### 4.5. P05: Comentarios y reseñas

| | |
|---|---|
| **Nombre** | P05: Comentarios y reseñas |
| **Descripción** | Participación en la conversación de una receta y calificación mediante estrellas. |
| **Actores** | Usuario registrado y autor del contenido. |
| **Entradas** | Texto, identificador de receta, referencia a la respuesta y puntuación. |

| | |
|---|---|
| **Salidas** | Comentario o reseña creada y promedio de calificación actualizado. |
| **Especificaciones de funcionamiento** | `POST /api/comments`, `GET /api/comments/receta/:recetaId`, `PUT /api/comments/:id` y `DELETE /api/comments/:id`; de forma análoga `POST /api/reviews`, `GET /api/reviews/receta/:recetaId`, `PUT /api/reviews/:id` y `DELETE /api/reviews/:id`. Un usuario deja una sola reseña por receta; el promedio se recalcula al crear, editar o borrar. |
| **Restricciones adicionales** | Texto no vacío, límite de creación de 10 por minuto, puntuación de 1 a 5 y control de autor. El borrado del comentario es lógico. Respuestas `201`, `200`, `400`, `404`, `429` y `500`. |

### 4.6. P06: Seguimiento, feed y notificaciones

| | |
|---|---|
| **Nombre** | P06: Seguimiento, feed y notificaciones |
| **Descripción** | Relaciones de seguimiento entre usuarios, descubrimiento de contenido mediante el feed y una campana provisional de notificaciones. |
| **Actores** | Usuario registrado. |
| **Entradas** | Identificador del usuario a seguir, número de página y tamaño de página. |
| **Salidas** | Estado de seguimiento, listas de seguidores y seguidos, feed paginado y campana de notificaciones. |

| | |
|---|---|
| **Especificaciones de funcionamiento** | `POST /api/follows`, `GET /api/follows/followers/:userId`, `GET /api/follows/following/:userId` y `GET /api/follows/check/:userId`; `GET /api/activity/feed?page=&limit=` y `GET /api/activity/user/:userId`. La campana reutiliza las actividades del feed. |
| **Restricciones adicionales** | Pareja única de seguimiento, límite de creación de 10 por minuto y 30 consultas por minuto. Defecto conocido: `/profile/:username` envía un nombre donde el backend espera un identificador, y la campana no persiste el estado leído. Respuestas `200`, `400`, `404`, `429` y `500`. |

## 5. Diagrama de flujo de los procesos

Los diagramas de flujo describen la secuencia de decisiones y actividades de los procesos principales del sistema. Las fuentes editables se encuentran en el directorio `docs/diagramas/flujo`.

## 5.1. Autenticación y sesión

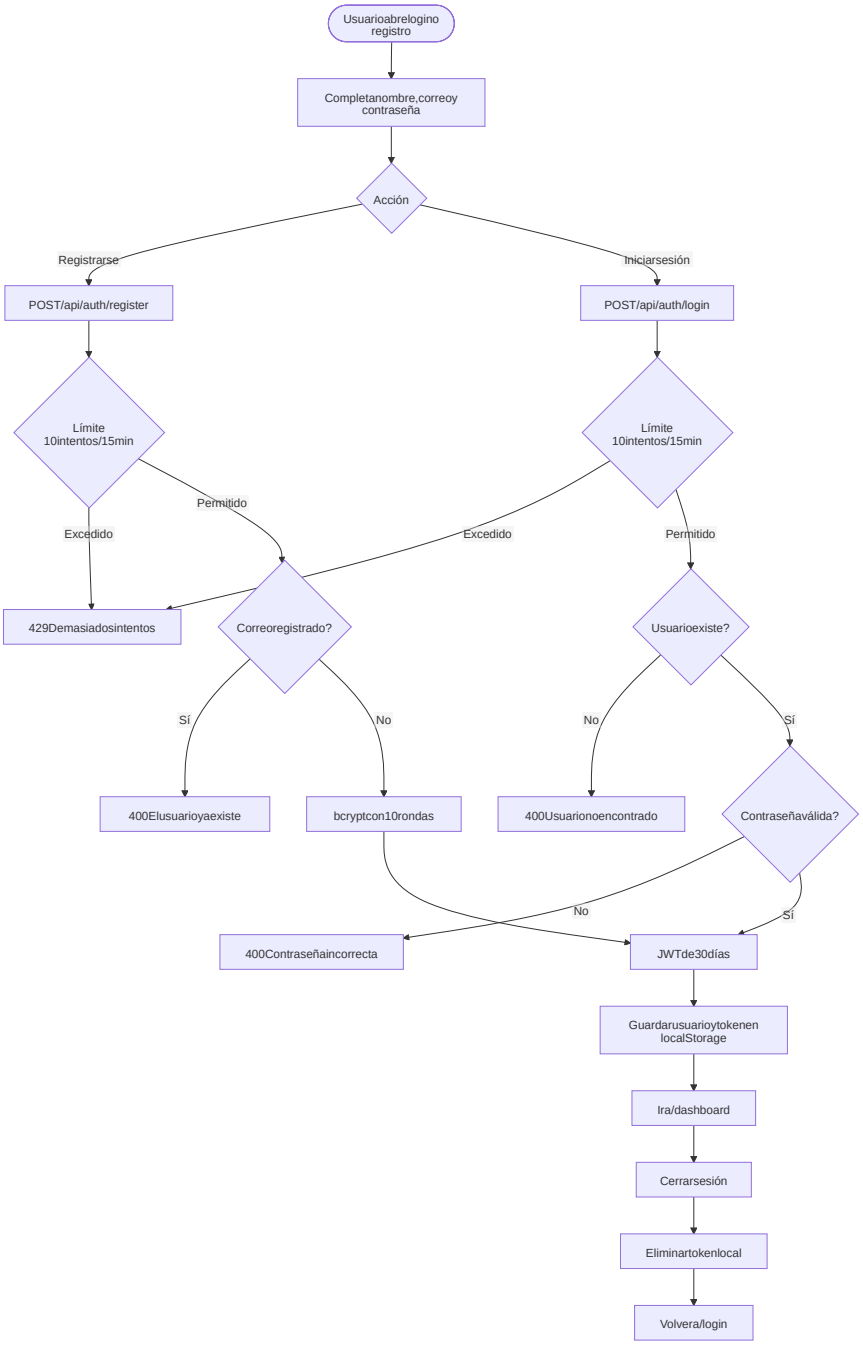



**Figura 1**

*Diagrama de flujo del proceso de registro e inicio de sesión.*

## 5.2. Recetas y visibilidad

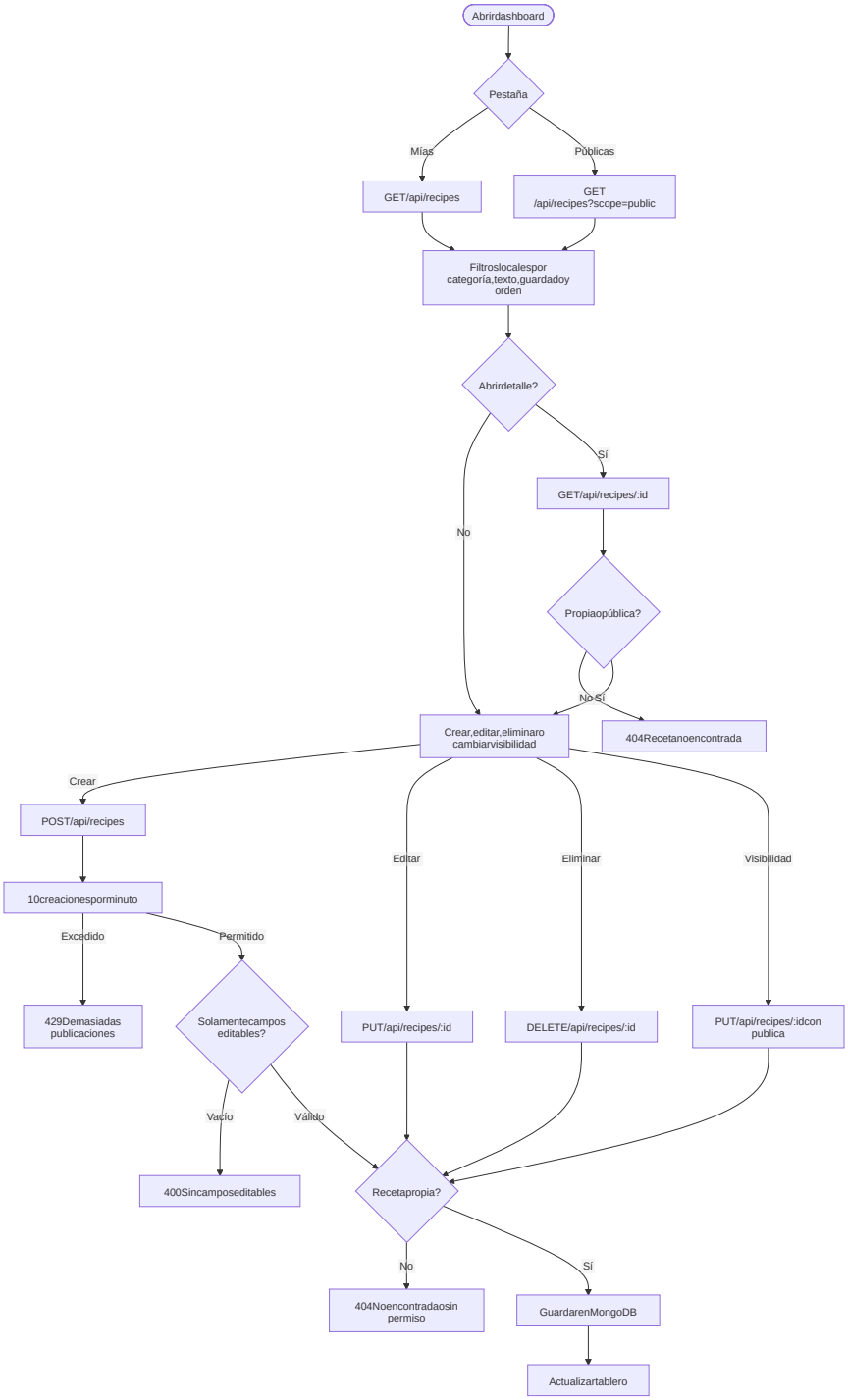



**Figura 2**

*Diagrama de flujo del proceso de creación, consulta y visibilidad de recetas.*

## 5.3. Imágenes

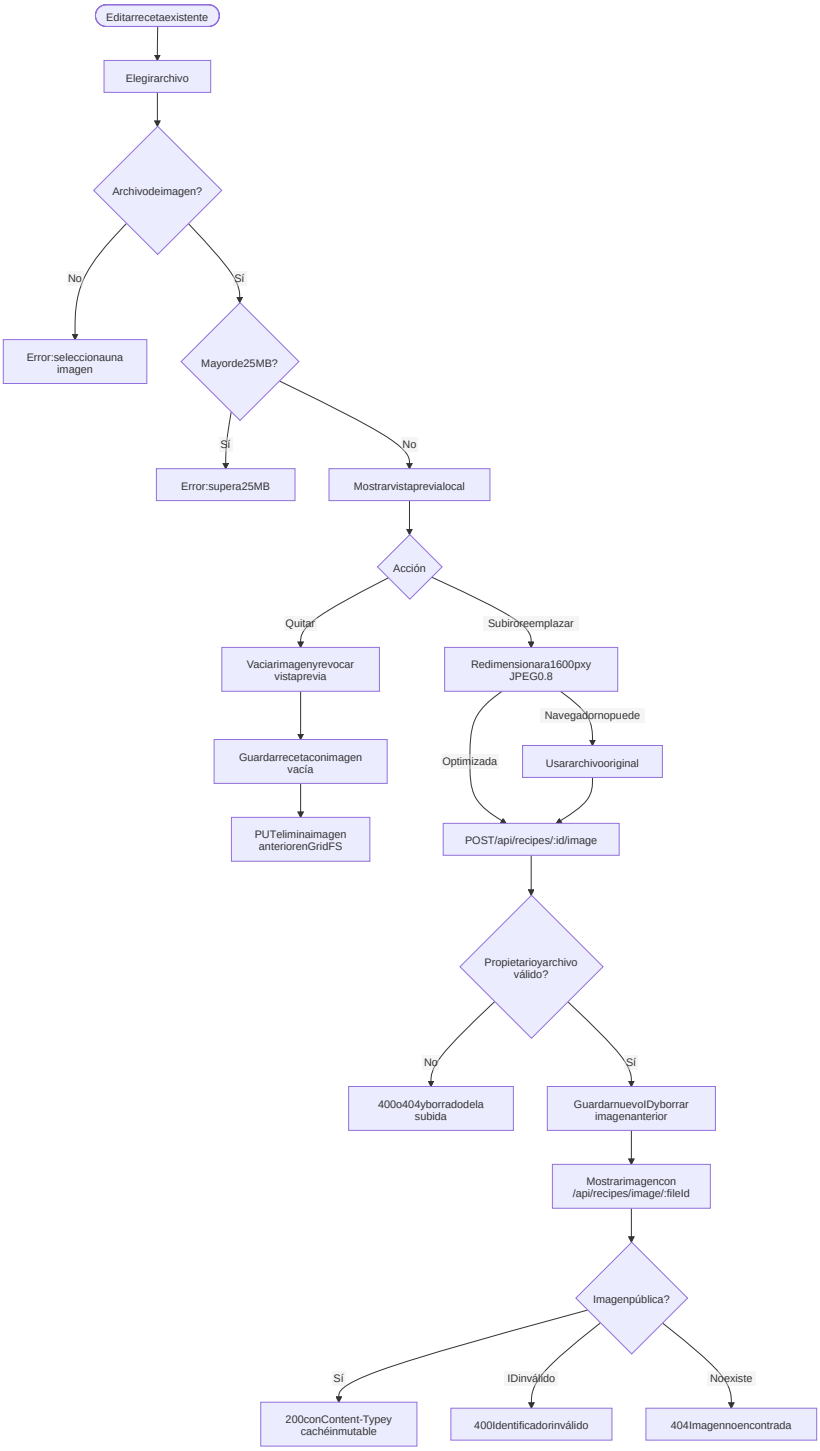



**Figura 3**

*Diagrama de flujo de la subida, reemplazo y eliminación de la imagen de una receta.*

## 5.4. Interacciones sociales

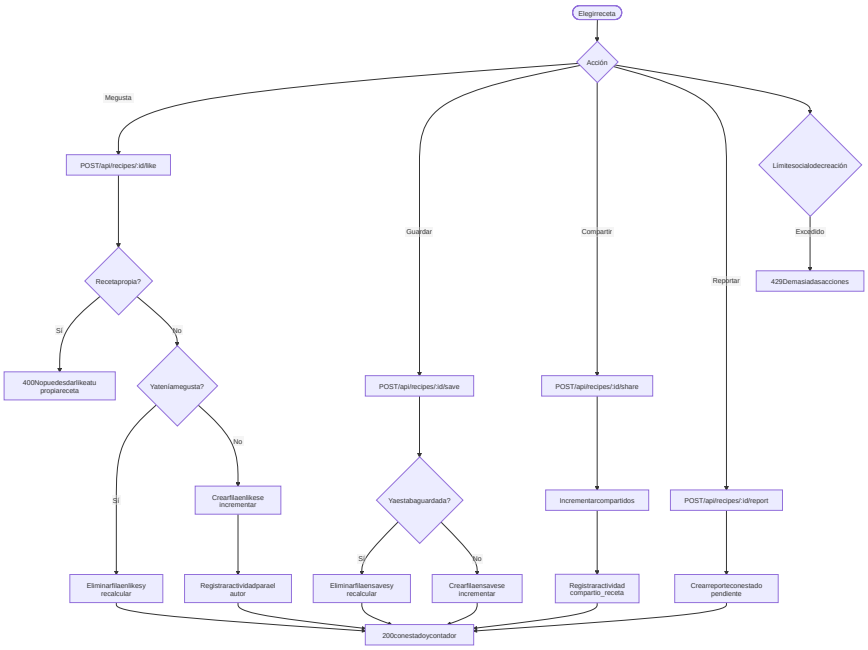



**Figura 4**

*Diagrama de flujo de las interacciones sociales sobre una receta.*

## 5.5. Comentarios y reseñas

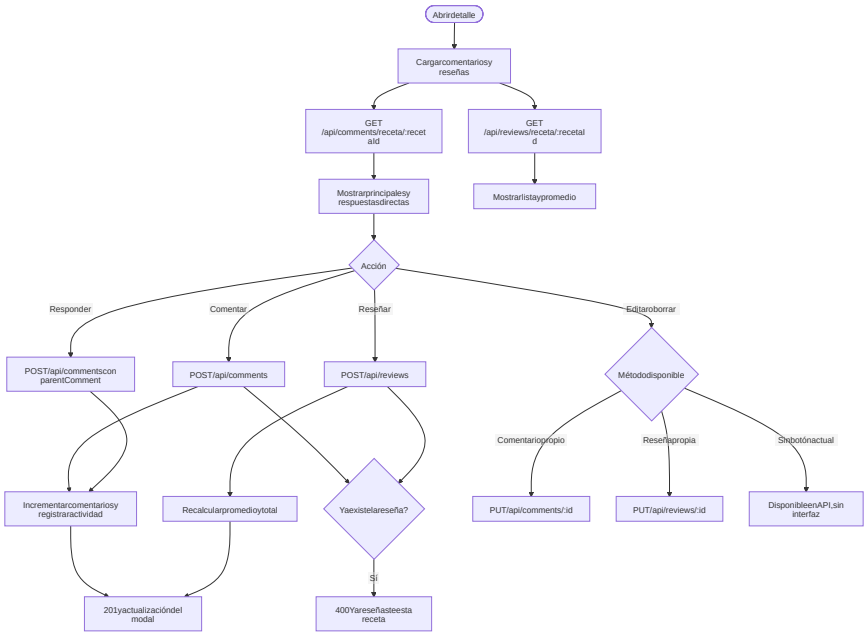



**Figura 5**

*Diagrama de flujo del proceso de comentarios y reseñas.*

## 5.6. Seguimiento, feed y notificaciones

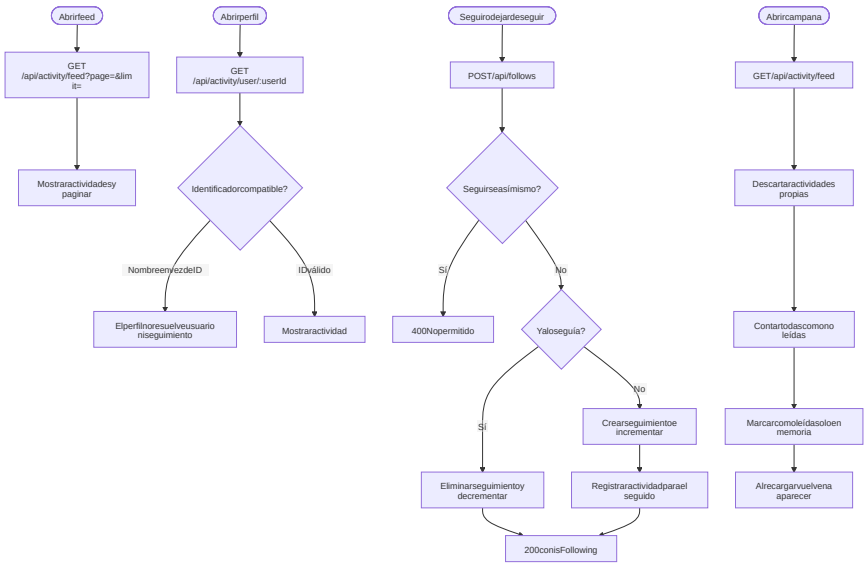



**Figura 6**

*Diagrama de flujo del seguimiento, el feed y la campana de notificaciones.*

## 5.7. Visión general del sistema

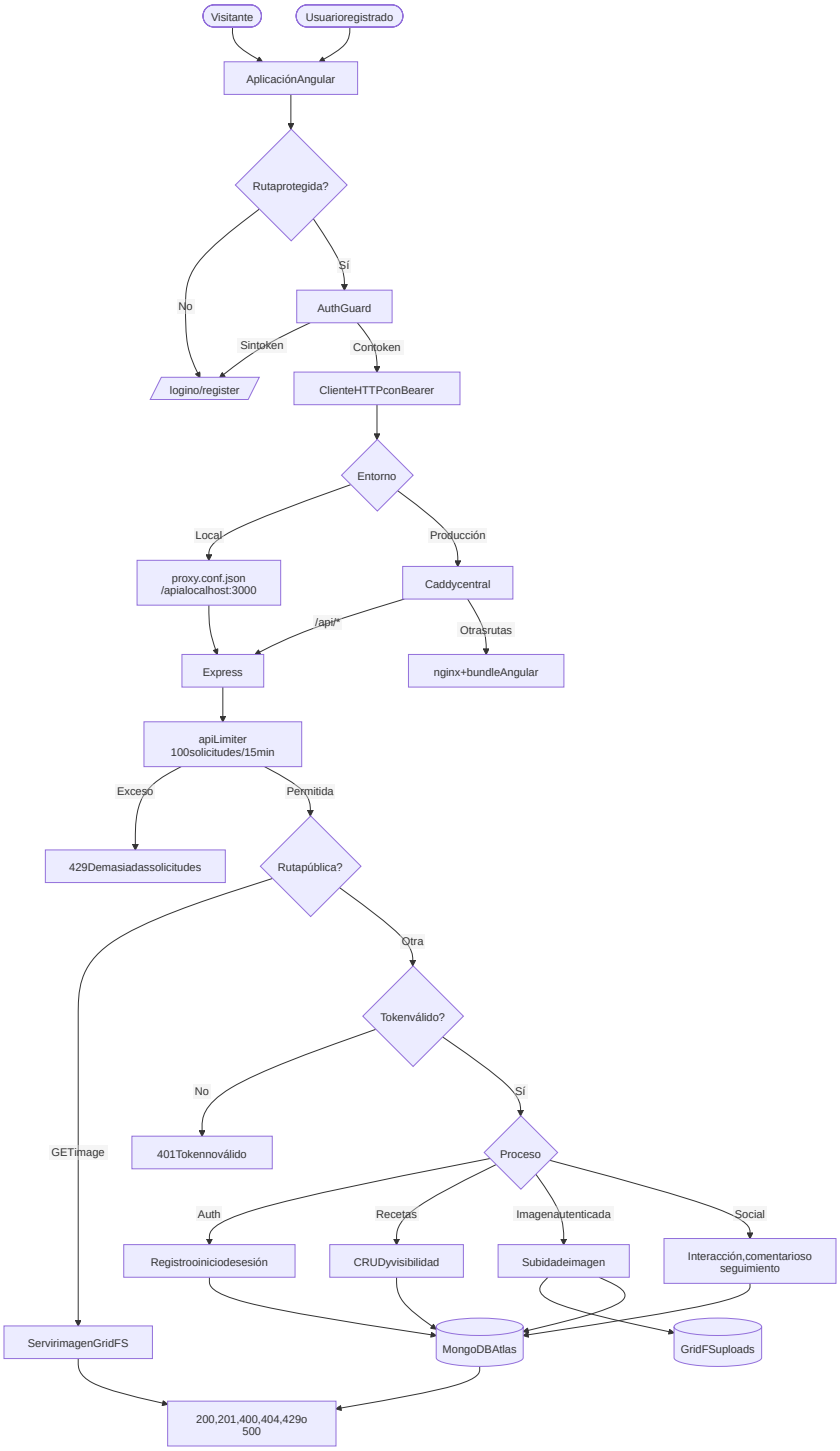



**Figura 7**

*Diagrama de flujo general del sistema FoodLoop.*

# 6. Casos de uso

Los casos de uso describen las interacciones entre los actores y el sistema por cada proceso. Cada módulo incluye su diagrama y las fichas narrativas correspondientes.

## 6.1. Casos de uso del módulo de autenticación

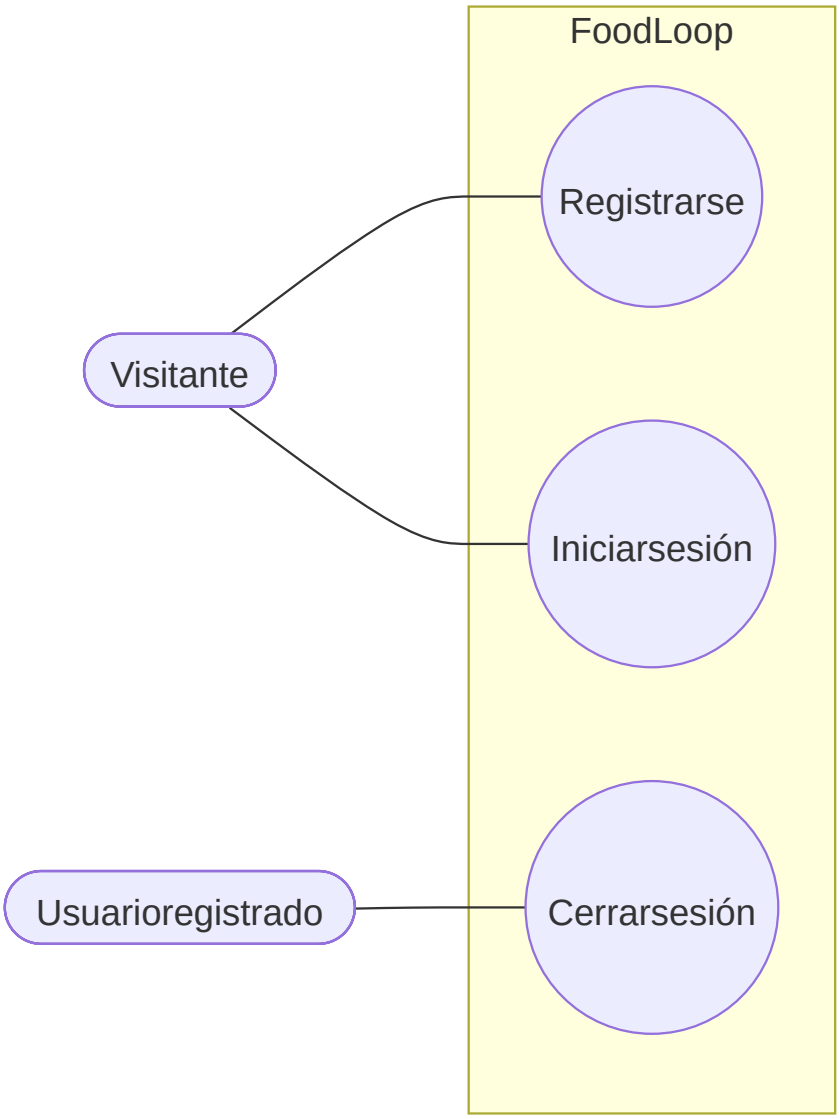



**Figura 8**

*Diagrama de casos de uso de autenticación y sesión.*

### *6.1.1. Caso de uso "CU-01: Registrarse"*

| | |
|---|---|
| **Descripción** | El visitante crea una cuenta con su nombre, correo y contraseña y recibe un token de sesión. |
| **Actores primarios** | Visitante. |
| **Contexto** | P01: Autenticación y sesión. |
| **Condiciones previas** | El correo no debe estar registrado. |
| **Flujo básico** | 1. El visitante abre el formulario de registro.<br>2. Ingresa su nombre, su correo y una contraseña de al menos 8 caracteres.<br>3. El sistema sanea los campos y comprueba que el correo no exista.<br>4. El sistema aplica el hash bcrypt a la contraseña y crea el usuario.<br>5. El sistema emite un token JWT y lo devuelve al cliente. |
| **Requerimientos especiales** | Contraseña de al menos 8 caracteres y correo único. |
| **Salida** | Usuario creado con respuesta `201` e ingreso al tablero. |
| **Excepciones** | Correo ya registrado (`400`); contraseña corta; exceso de intentos (`429`). |

### *6.1.2. Caso de uso "CU-02: Iniciar sesión"*

| | |
|---|---|
| **Descripción** | El usuario autenticado accede a la aplicación con sus credenciales. |
| **Actores primarios** | Visitante. |
| **Contexto** | P01: Autenticación y sesión. |

| | |
|---|---|
| **Condiciones previas** | El usuario debe estar registrado. |
| **Flujo básico** | 1. El usuario ingresa su correo y su contraseña.<br>2. El sistema busca el usuario y compara el hash de la contraseña.<br>3. Si coinciden, el sistema emite un token JWT de 30 días.<br>4. El cliente almacena el token y accede al tablero. |
| **Requerimientos especiales** | Límite de 10 intentos cada 15 minutos. |
| **Salida** | Sesión iniciada con respuesta `200`. |
| **Excepciones** | Usuario o contraseña incorrectos (`400`); exceso de intentos (`429`). |

### *6.1.3. Caso de uso "CU-03: Cerrar sesión"*

| | |
|---|---|
| **Descripción** | El usuario registrado termina su sesión en el navegador. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P01: Autenticación y sesión. |
| **Condiciones previas** | El usuario debe haber iniciado sesión. |
| **Flujo básico** | 1. El usuario pulsa la opción de cerrar sesión.<br>2. El cliente elimina el token local.<br>3. El cliente redirige a la pantalla de inicio de sesión. |

| | |
|---|---|
| **Requerimientos especiales** | El cierre de sesión no invalida el token en el servidor. |
| **Salida** | Sesión cerrada localmente. |
| **Excepciones** | Un token copiado sigue siendo válido hasta su expiración. |

## 6.2. Casos de uso del módulo de recetas

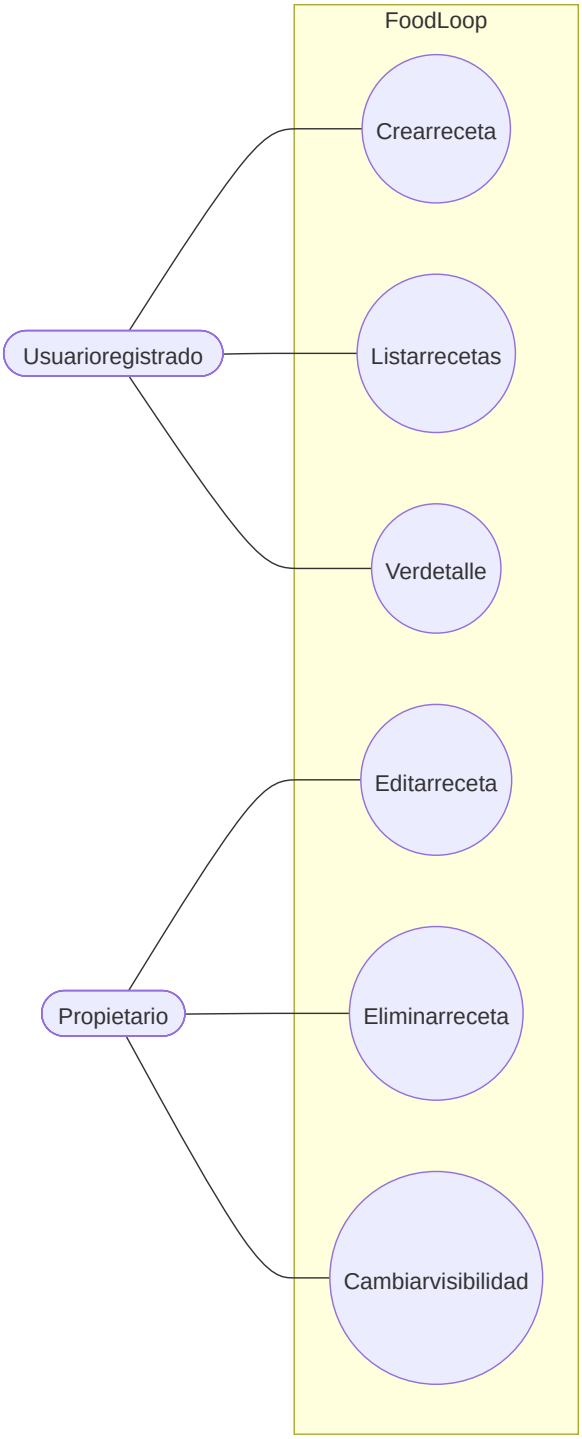



**Figura 9**

*Diagrama de casos de uso de recetas y visibilidad.*

### *6.2.1. Caso de uso "CU-04: Crear receta"*

| | |
|---|---|
| **Descripción** | El usuario guarda una receta propia. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P02: Gestión de recetas y visibilidad. |
| **Condiciones previas** | El usuario debe tener una sesión válida. |
| **Flujo básico** | 1. El usuario completa el formulario de receta.<br>2. El sistema conserva únicamente los campos editables y asigna el propietario.<br>3. La receta se guarda en estado privado.<br>4. El sistema confirma la creación con la receta creada. |
| **Requerimientos especiales** | Límite de 10 creaciones por minuto; la receta nace privada. |
| **Salida** | Receta creada con respuesta `201`. |
| **Excepciones** | Exceso de creaciones (`429`); error de validación. |

### *6.2.2. Caso de uso "CU-05: Listar y ver recetas"*

| | |
|---|---|
| **Descripción** | El usuario consulta sus recetas o las recetas públicas de otros usuarios. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P02: Gestión de recetas y visibilidad. |
| **Condiciones previas** | El usuario debe tener una sesión válida. |

| | |
|---|---|
| **Flujo básico** | 1. El tablero solicita las recetas propias o, con `scope=public`, las públicas.<br>2. El sistema devuelve la lista correspondiente.<br>3. Al abrir el detalle, el sistema permite la receta propia o una pública. |
| **Requerimientos especiales** | Sin paginación ni búsqueda en el servidor. |
| **Salida** | Lista y detalle mostrados. |
| **Excepciones** | Receta privada ajena (`404`); identificador inválido puede producir error. |

### *6.2.3. Caso de uso "CU-06: Editar, eliminar y cambiar visibilidad"*

| | |
|---|---|
| **Descripción** | El propietario actualiza, borra o publica una receta. |
| **Actores primarios** | Propietario de receta. |
| **Contexto** | P02: Gestión de recetas y visibilidad. |
| **Condiciones previas** | El usuario debe ser propietario de la receta. |
| **Flujo básico** | 1. El propietario guarda los cambios, confirma el borrado o alterna la visibilidad.<br>2. El sistema valida la propiedad y aplica la lista de campos editables.<br>3. El sistema ejecuta los validadores y persiste el cambio. |

| | |
|---|---|
| **Requerimientos especiales** | El borrado es físico y deja huérfanos los comentarios, reseñas y actividades asociados. |
| **Salida** | Receta actualizada, eliminada o publicada. |
| **Excepciones** | Sin campos editables (400); receta ajena o inexistente (404). |

### 6.3. Casos de uso del módulo de imágenes

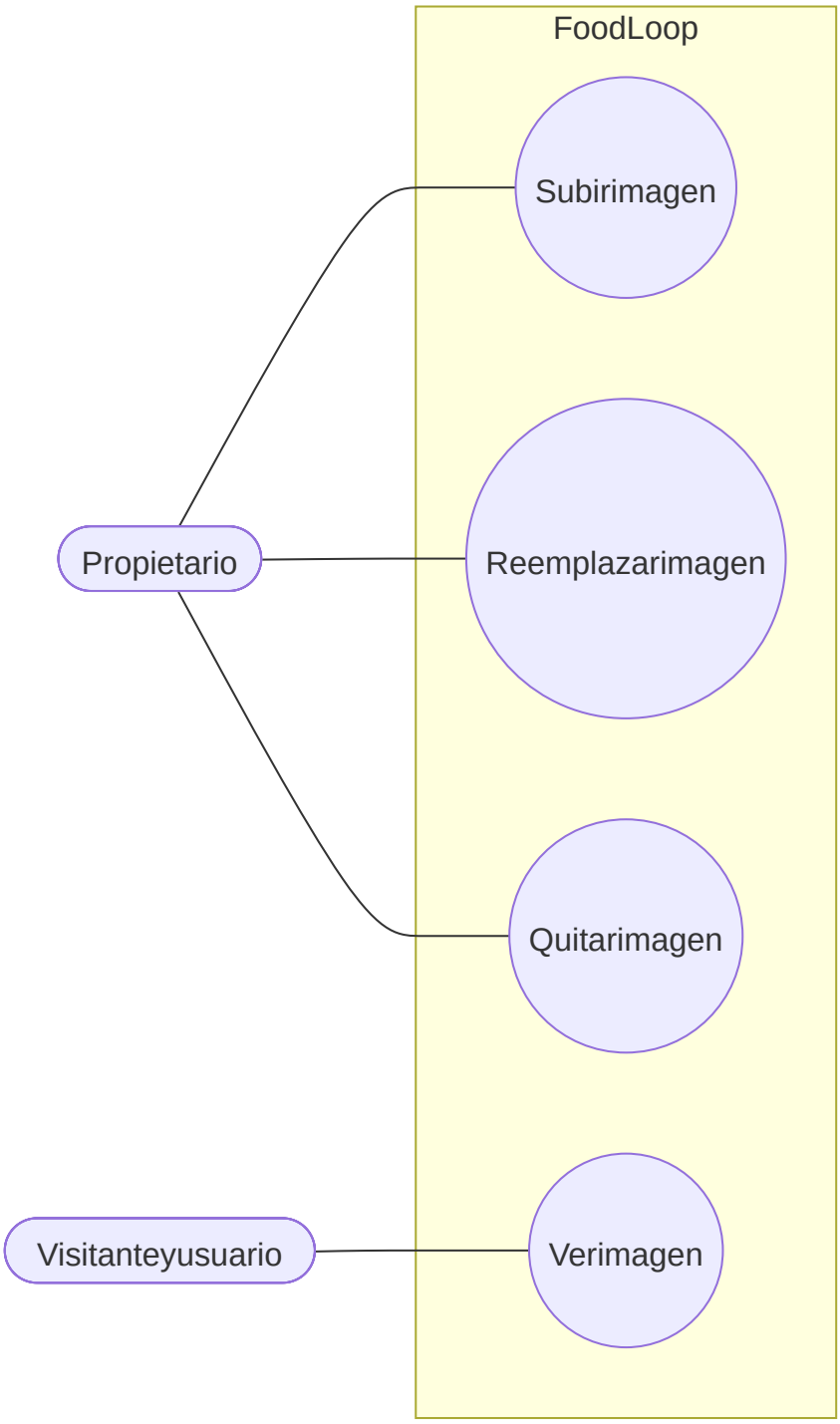



**Figura 10**

*Diagrama de casos de uso de imágenes.*

#### *6.3.1. Caso de uso "CU-07: Subir imagen"*

| | |
|---|---|
| **Descripción** | El propietario asocia una imagen optimizada a una receta existente. |
| **Actores primarios** | Propietario de receta. |
| **Contexto** | P03: Gestión de imágenes. |
| **Condiciones previas** | La receta debe existir y pertenecer al usuario. |
| **Flujo básico** | 1. El usuario selecciona un archivo de imagen.<br>2. El navegador valida el tipo y optimiza la imagen a 1600 px y JPEG de calidad 0.8.<br>3. El archivo se envía como `multipart/form-data`.<br>4. El servidor guarda la imagen en GridFS, actualiza el identificador de la receta y elimina la imagen anterior. |
| **Requerimientos especiales** | MIME de imagen, 25 MB en el navegador y 5 MB en el servidor. |
| **Salida** | Receta con imagen actualizada (`200`). |
| **Excepciones** | Tipo no permitido o tamaño excedido (`400`); receta ajena (`404`). |

### *6.3.2. Caso de uso "CU-08: Reemplazar o quitar imagen"*

| | |
|---|---|
| **Descripción** | El propietario mantiene una sola imagen vigente o deja la receta sin imagen. |
| **Actores primarios** | Propietario de receta. |
| **Contexto** | P03: Gestión de imágenes. |
| **Condiciones previas** | El usuario debe ser propietario de la receta. |

| | |
|---|---|
| **Flujo básico** | 1. El propietario elige otra imagen o elimina la actual.<br>2. El sistema registra el nuevo identificador o una cadena vacía.<br>3. El sistema elimina de GridFS el archivo anterior. |
| **Requerimientos especiales** | Solo se conserva una imagen vigente por receta. |
| **Salida** | Imagen reemplazada o eliminada. |
| **Excepciones** | Un fallo de subida conserva el identificador anterior. |

### *6.3.3. Caso de uso "CU-09: Ver imagen"*

| | |
|---|---|
| **Descripción** | El sistema muestra la imagen en tarjetas, detalle o enlace directo. |
| **Actores primarios** | Cualquier persona que conozca la URL pública. |
| **Contexto** | P03: Gestión de imágenes. |
| **Condiciones previas** | Se debe conocer el identificador del archivo. |
| **Flujo básico** | 1. El cliente solicita la imagen por su identificador.<br>2. El sistema valida el identificador y lee el archivo desde GridFS.<br>3. El sistema devuelve el contenido con su tipo y aplica caché inmutable. |
| **Requerimientos especiales** | Lectura pública, sin cabecera de autorización. |
| **Salida** | Imagen servida con respuesta `200`. |
| **Excepciones** | Identificador inválido (`400`); archivo inexistente (`404`). |

## 6.4. Casos de uso del módulo de interacciones sociales

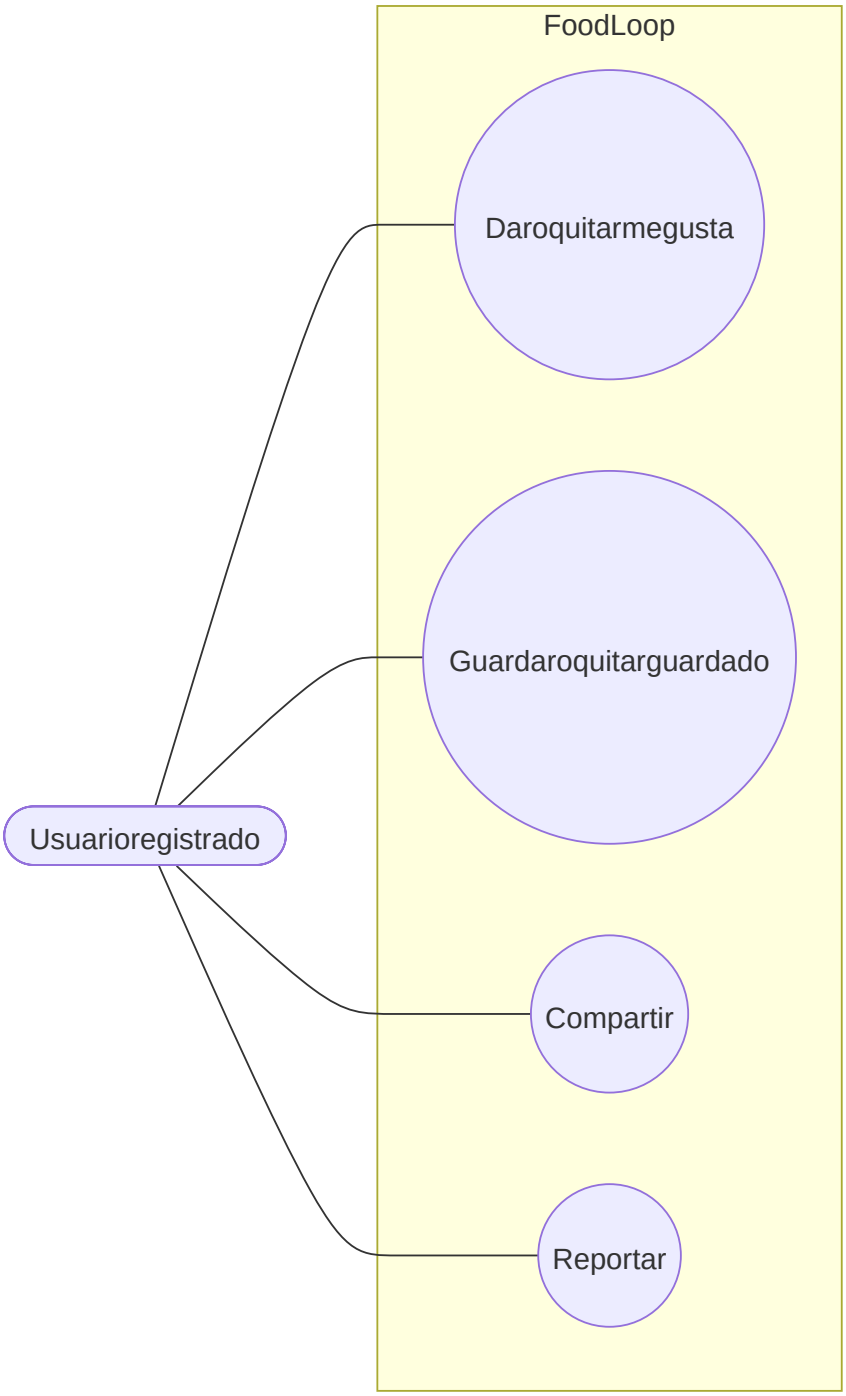



**Figura 11**

*Diagrama de casos de uso de interacciones sociales.*

### *6.4.1. Caso de uso "CU-10: Me gusta"*

| | |
|---|---|
| **Descripción** | El usuario marca o desmarca una receta con un me gusta. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P04: Interacciones sociales. |
| **Condiciones previas** | El usuario debe tener una sesión válida. |
| **Flujo básico** | 1. El usuario pulsa el corazón de una receta.<br>2. El sistema crea o elimina la fila en `likes` y recalcula el contador.<br>3. El sistema registra la actividad `likio_receta` para el autor. |
| **Requerimientos especiales** | Límite social de 30 acciones por minuto. |
| **Salida** | Estado alternado con `liked` y `likesCount`. |
| **Excepciones** | Receta inexistente (`404`); dar me gusta a la receta propia (`400`). |

### *6.4.2. Caso de uso "CU-10b: Guardar receta"*

| | |
|---|---|
| **Descripción** | El usuario guarda o quita una receta de su lista de guardados. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P04: Interacciones sociales. |
| **Condiciones previas** | El usuario debe tener una sesión válida. |

| | |
|---|---|
| **Flujo básico** | 1. El usuario pulsa el botón de guardar.<br>2. El sistema crea o elimina la fila en `saves` y recalcula el contador de guardados de la receta. |
| **Requerimientos especiales** | Es el único mecanismo de favorito personal. |
| **Salida** | Estado alternado con `saved` y `guardadosCount`. |
| **Excepciones** | Receta inexistente (`404`). |

### *6.4.3. Caso de uso "CU-10c: Compartir receta"*

| | |
|---|---|
| **Descripción** | El usuario comparte una receta, lo que incrementa su contador de compartidos. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P04: Interacciones sociales. |
| **Condiciones previas** | El usuario debe tener una sesión válida. |
| **Flujo básico** | 1. El usuario pulsa la opción de compartir.<br>2. El sistema incrementa el contador de compartidos y registra la actividad `compartio_receta`. |
| **Requerimientos especiales** | Límite social de 30 acciones por minuto; `compartio_receta` es un tipo válido de actividad. |
| **Salida** | Contador incrementado (`compartidosCount`). |
| **Excepciones** | Receta inexistente (`404`). |

### *6.4.4. Caso de uso "CU-10d: Reportar receta"*

| | |
|---|---|
| **Descripción** | El usuario informa contenido inapropiado indicando un motivo. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P04: Interacciones sociales. |
| **Condiciones previas** | El usuario debe tener una sesión válida. |
| **Flujo básico** | 1. El usuario envía un motivo de reporte.<br>2. El sistema sanea el motivo y crea un documento en `reports` con estado `pendiente`. |
| **Requerimientos especiales** | Un usuario reporta una receta una sola vez; no existe moderación ni cuenta administradora. |
| **Salida** | Reporte creado con estado `pendiente`. |
| **Excepciones** | Receta inexistente (`404`); reporte duplicado del mismo usuario (`400`). |

## 6.5. Casos de uso del módulo de comentarios y reseñas

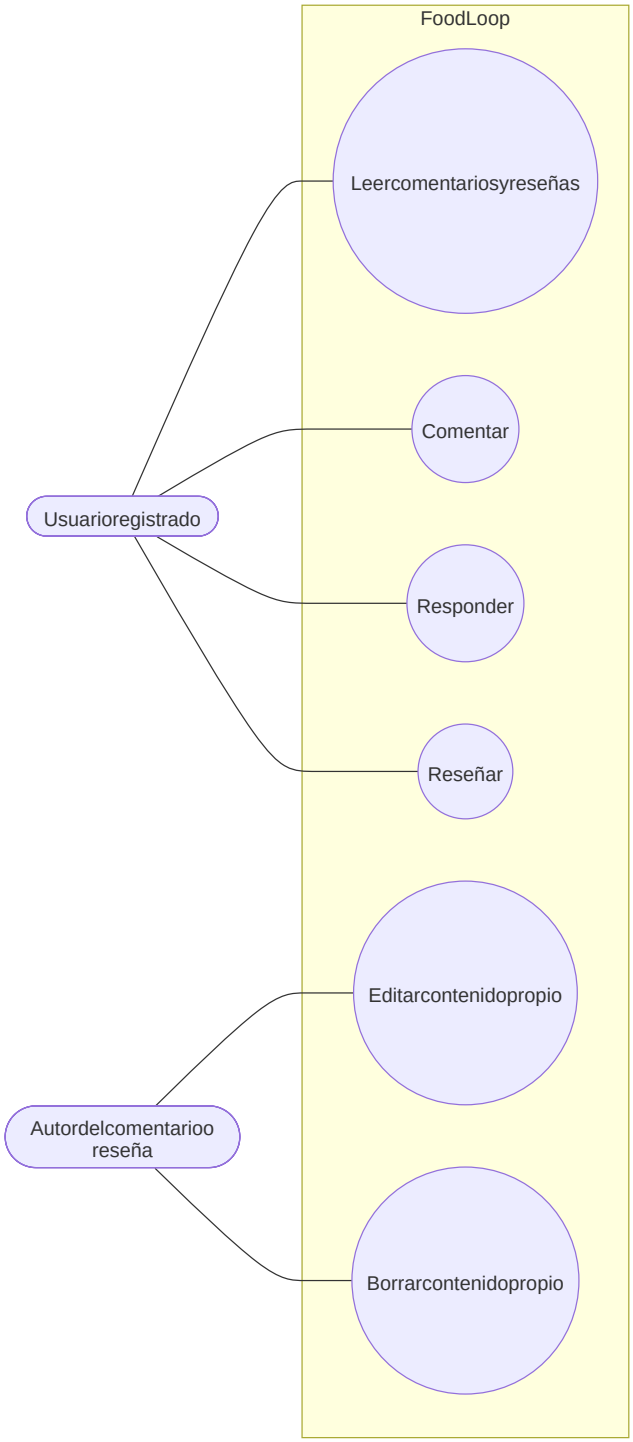



**Figura 12**

*Diagrama de casos de uso de comentarios y reseñas.*

### *6.5.1. Caso de uso "CU-11: Leer comentarios y reseñas"*

| | |
|---|---|
| **Descripción** | El usuario consulta la conversación y la calificación de una receta. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P05: Comentarios y reseñas. |
| **Condiciones previas** | El usuario debe tener una sesión válida. |
| **Flujo básico** | 1. El usuario abre el detalle de la receta.<br>2. El sistema solicita los comentarios principales con un nivel de respuestas y las reseñas ordenadas.<br>3. El sistema muestra los comentarios, las reseñas y el promedio de calificación. |
| **Requerimientos especiales** | La lectura expone solo un nivel de respuestas. |
| **Salida** | Comentarios, reseñas y promedio mostrados. |
| **Excepciones** | Los errores de lectura muestran mensajes genéricos. |

### *6.5.2. Caso de uso "CU-12: Comentar y responder"*

| | |
|---|---|
| **Descripción** | El usuario publica un comentario o una respuesta de primer nivel. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P05: Comentarios y reseñas. |
| **Condiciones previas** | Sesión válida y texto no vacío. |

| | |
|---|---|
| **Flujo básico** | 1. El usuario escribe el comentario o la respuesta.<br>2. El sistema lo guarda, incrementa el contador de la receta y registra la actividad.<br>3. El comentario se muestra en el detalle. |
| **Requerimientos especiales** | Límite de 10 creaciones por minuto; el borrado es lógico. |
| **Salida** | Comentario creado con respuesta `201`. |
| **Excepciones** | Texto vacío o excesivo; no se comprueba que la receta exista o sea visible. |

### *6.5.3. Caso de uso "CU-13: Reseñar"*

| | |
|---|---|
| **Descripción** | El usuario califica una receta de 1 a 5 estrellas, con un texto opcional. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P05: Comentarios y reseñas. |
| **Condiciones previas** | Sesión válida y puntuación distinta de cero. |
| **Flujo básico** | 1. El usuario envía su calificación y, opcionalmente, un texto.<br>2. El sistema guarda la reseña y recalcula el promedio y el total de la receta.<br>3. El sistema registra la actividad y muestra la reseña. |

| | |
|---|---|
| **Requerimientos especiales** | Una sola reseña por usuario y receta; promedio redondeado a un decimal. |
| **Salida** | Reseña creada con respuesta `201` y promedio actualizado. |
| **Excepciones** | Segunda reseña del mismo usuario (`400`); puntuación inválida puede producir `500`. |

### *6.5.4. Caso de uso "CU-14: Editar o borrar contenido propio"*

| | |
|---|---|
| **Descripción** | El autor corrige o retira su propio comentario o reseña. |
| **Actores primarios** | Autor del contenido. |
| **Contexto** | P05: Comentarios y reseñas. |
| **Condiciones previas** | El usuario debe ser el autor del contenido. |
| **Flujo básico** | 1. El autor actualiza el texto o solicita el borrado lógico.<br>2. El sistema verifica la autoría y recalcula las métricas. |
| **Requerimientos especiales** | La operación existe en la API, pero aún no tiene botones en la interfaz. |
| **Salida** | Comentario o reseña actualizado o retirado. |
| **Excepciones** | Control de autor; recurso inexistente (`404`). |

## 6.6. Casos de uso del módulo de seguimiento, feed y notificaciones

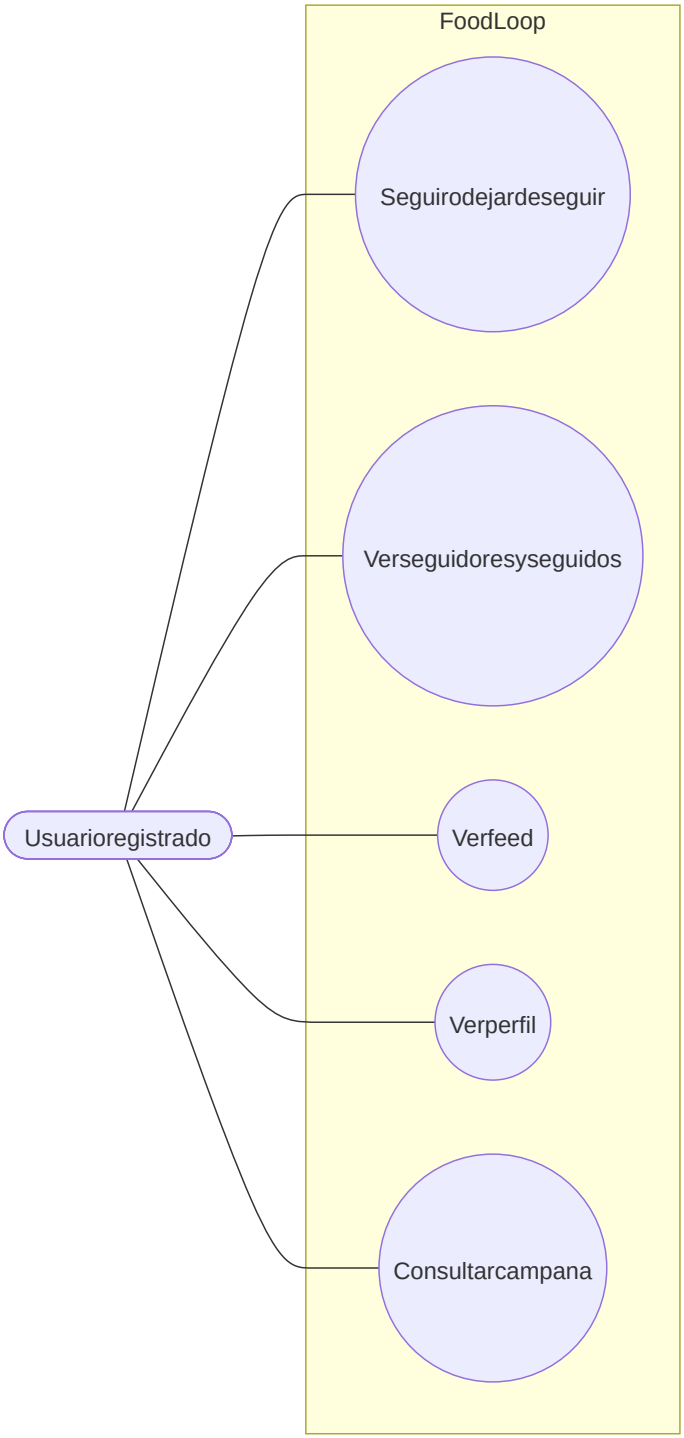



**Figura 13**

*Diagrama de casos de uso de seguimiento, feed y notificaciones.*

### *6.6.1. Caso de uso "CU-14b: Seguir o dejar de seguir"*

| | |
|---|---|
| **Descripción** | El usuario se suscribe o cancela la suscripción a otro usuario. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P06: Seguimiento, feed y notificaciones. |
| **Condiciones previas** | Sesión válida y un destinatario distinto del propio usuario. |
| **Flujo básico** | 1. El usuario pulsa el botón de seguir en el perfil de otro usuario.<br>2. El sistema alterna el documento de seguimiento.<br>3. El sistema actualiza los contadores de seguidores y seguidos y registra la actividad. |
| **Requerimientos especiales** | Pareja única de seguimiento; límite de 10 creaciones por minuto. |
| **Salida** | Estado alternado con `isFollowing`. |
| **Excepciones** | El usuario intenta seguirse a sí mismo (`400`). |

### *6.6.2. Caso de uso "CU-15: Ver seguidores, seguidos y feed"*

| | |
|---|---|
| **Descripción** | El usuario explora sus relaciones sociales y la actividad de las personas que sigue. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P06: Seguimiento, feed y notificaciones. |
| **Condiciones previas** | El usuario debe tener una sesión válida. |

| | |
|---|---|
| **Flujo básico** | 1. El usuario abre su perfil o su feed.<br>2. El sistema devuelve los seguidores, los seguidos y las actividades paginadas.<br>3. El sistema muestra las listas y el feed. |
| **Requerimientos especiales** | Defecto conocido: `/profile/:username` envía un nombre donde el backend espera un identificador. |
| **Salida** | Listas y feed mostrados. |
| **Excepciones** | El perfil no resuelve correctamente el usuario, sus recetas ni su seguimiento. |

### *6.6.3. Caso de uso "CU-16: Consultar la campana de notificaciones"*

| | |
|---|---|
| **Descripción** | El usuario consulta la campana de notificaciones con la actividad reciente. |
| **Actores primarios** | Usuario registrado. |
| **Contexto** | P06: Seguimiento, feed y notificaciones. |
| **Condiciones previas** | El usuario debe tener una sesión válida. |
| **Flujo básico** | 1. El usuario abre la campana de notificaciones.<br>2. El sistema reutiliza las actividades del feed y descarta las propias.<br>3. El sistema muestra las notificaciones y su contador. |

| | |
|---|---|
| **Requerimientos especiales** | La campana no persiste el estado leído y cuenta todas las actividades como nuevas. |
| **Salida** | Campana de notificaciones mostrada. |
| **Excepciones** | Pendiente la persistencia de notificaciones (prevista para un ciclo posterior). |

## 7. Diagrama de secuencia

Los diagramas de secuencia muestran el intercambio de mensajes entre los componentes en los procesos con mayor interacción. Las fuentes editables se encuentran en el directorio `docs/diagramas/secuencia`.

## 7.1. Secuencia de registro e inicio de sesión

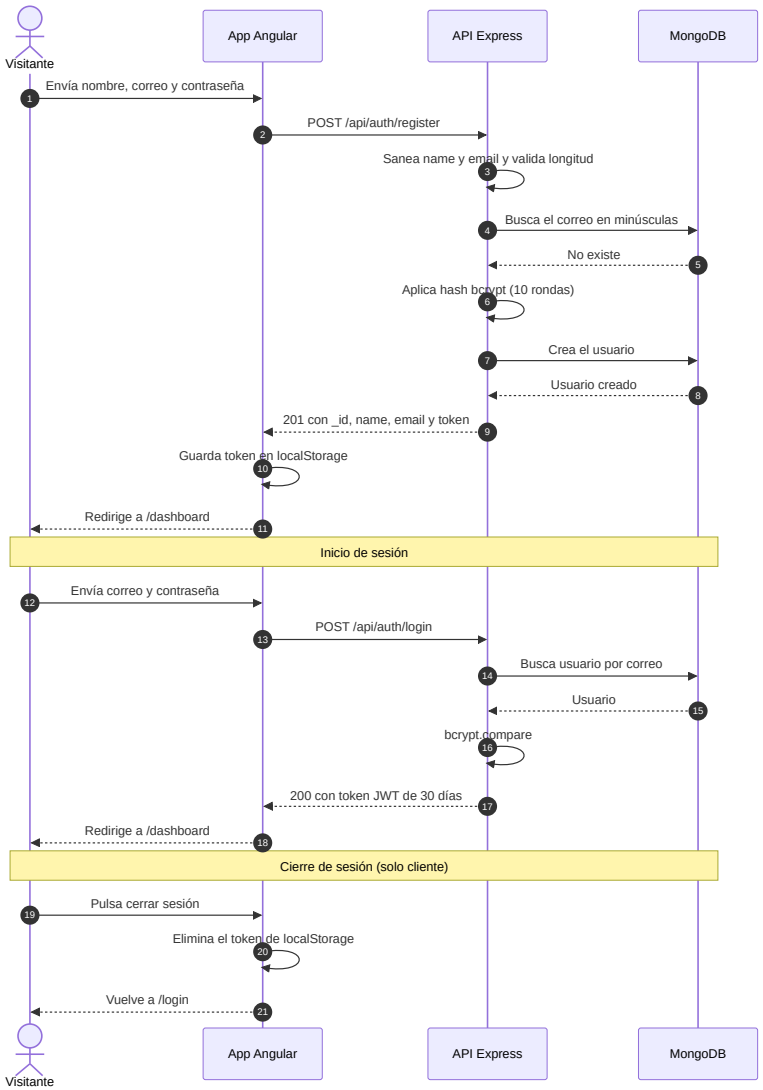



**Figura 14**

*Secuencia de registro e inicio de sesión.*

## 7.2. Secuencia de creación de receta y gestión de imagen

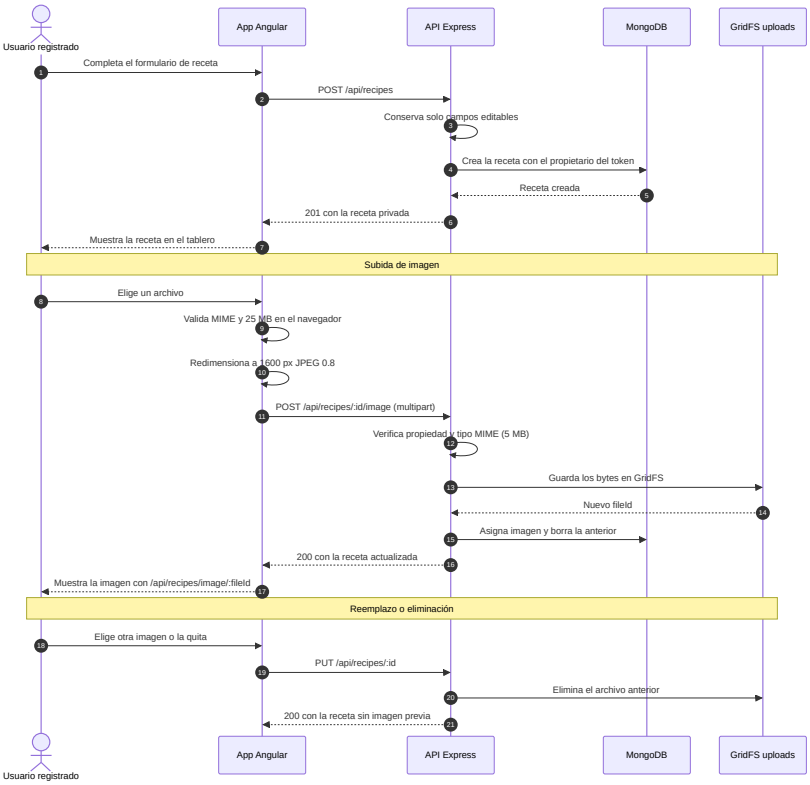



**Figura 15**

*Secuencia de creación de una receta y subida de su imagen.*

## 7.3. Secuencia de comentar y reseñar

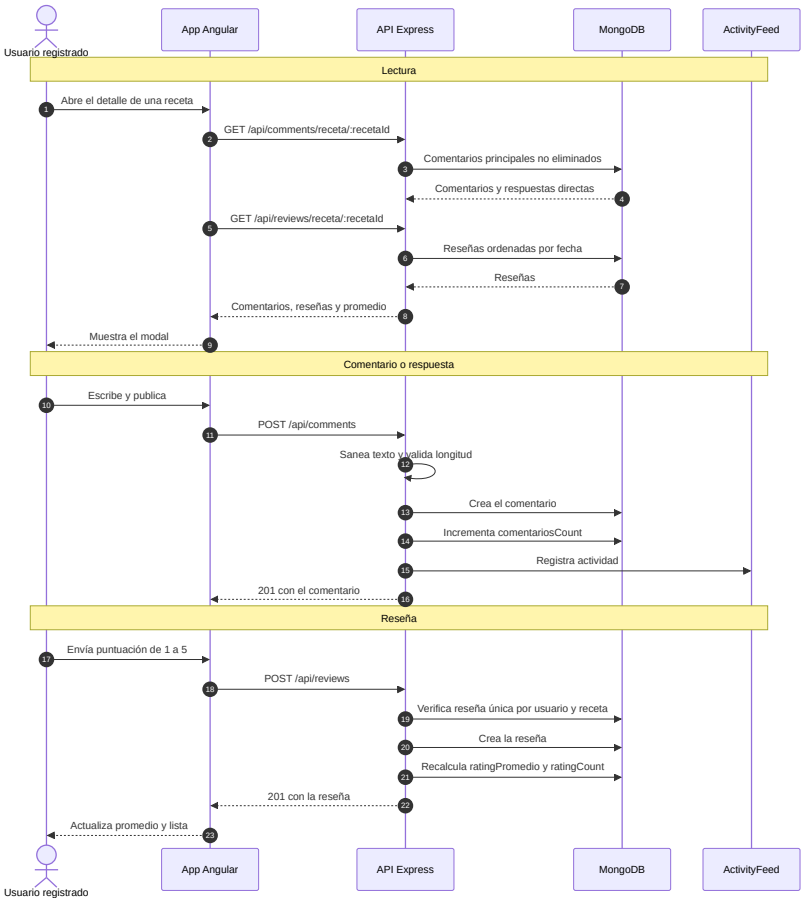



**Figura 16**

*Secuencia de publicación de un comentario y de una reseña.*

### 7.4. Secuencia de seguir usuarios y consultar el feed

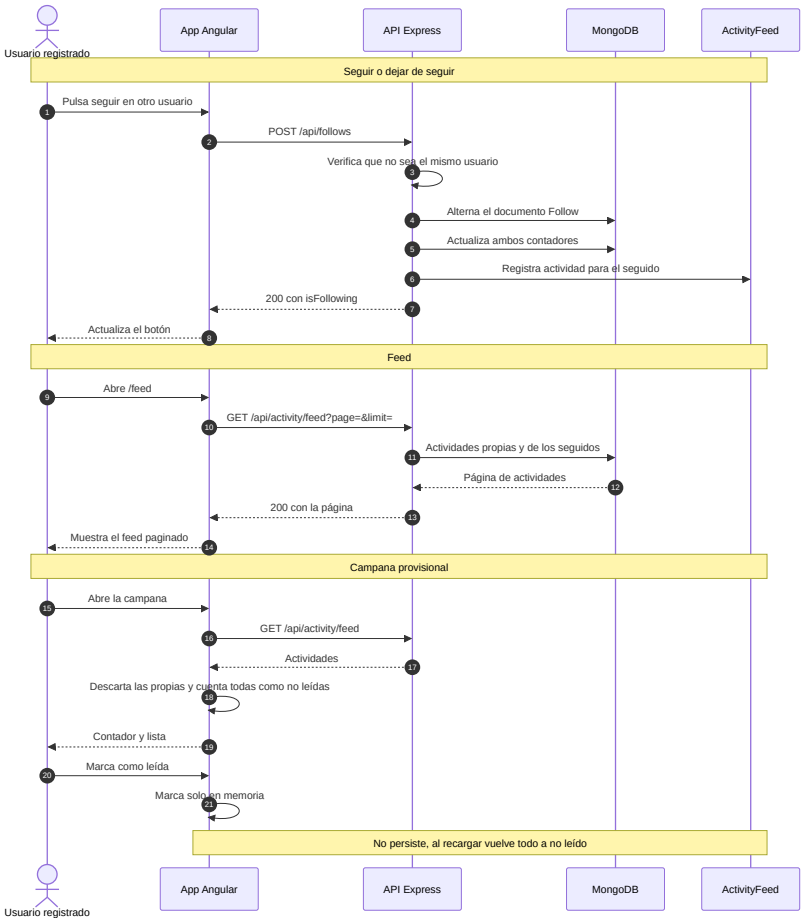



**Figura 17**

*Secuencia del seguimiento entre usuarios y de la consulta del feed.*

## 8. Diagramas de estado

Los diagramas de estado describen el ciclo de vida de los recursos con estados propios. Las fuentes editables se encuentran en el directorio `docs/diagramas/estado`.

### 8.1. Estados de la receta

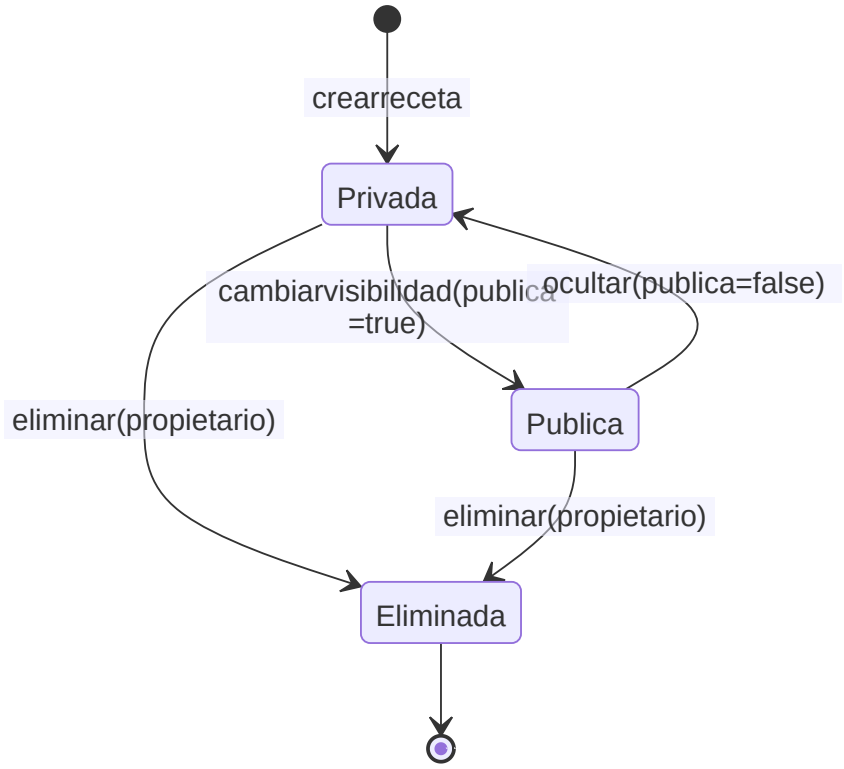



**Figura 18**

*Estados de una receta: visibilidad y eliminación.*

La receta nace privada. La visibilidad es un atributo del documento y el borrado es físico: no hay borrado lógico ni eliminación en cascada de comentarios, reseñas y actividades. Los reportes son documentos aparte en `reports` y no cambian la visibilidad ni el estado de la receta; al eliminar una receta se borran también sus filas en `likes`, `saves` y `reports`.

### 8.2. Estados de la imagen

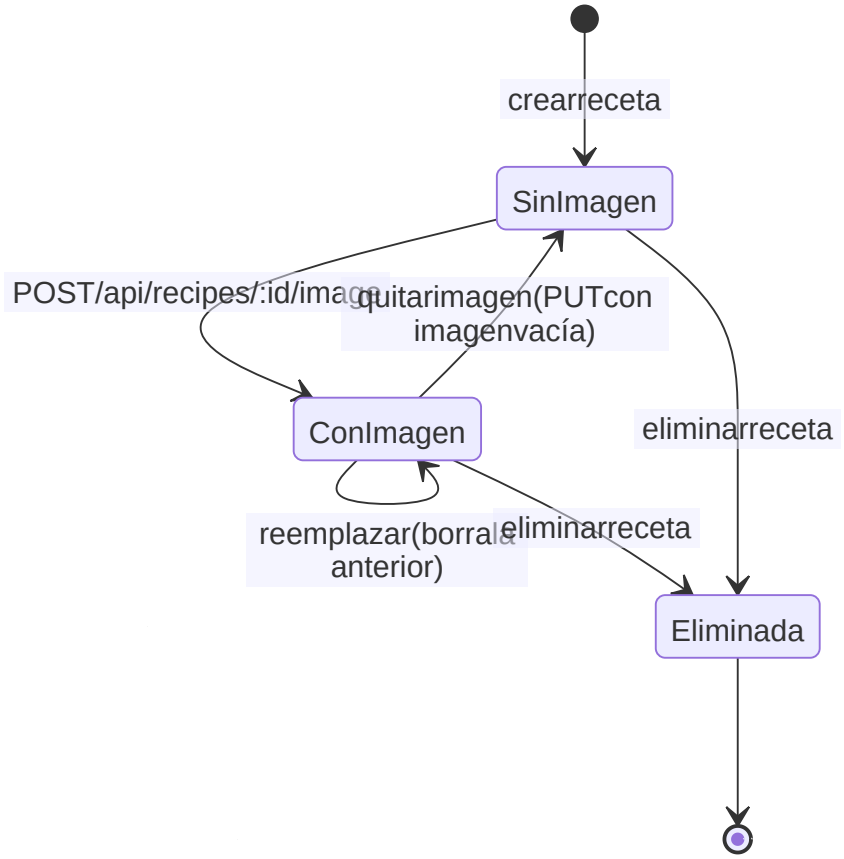



**Figura 19**

*Estados de la imagen de una receta.*

Solo se conserva una imagen vigente por receta. El archivo anterior se elimina de GridFS al reemplazar, quitar o borrar la receta. La lectura es pública y usa caché inmutable.

### 8.3. Estados de la campana de notificaciones

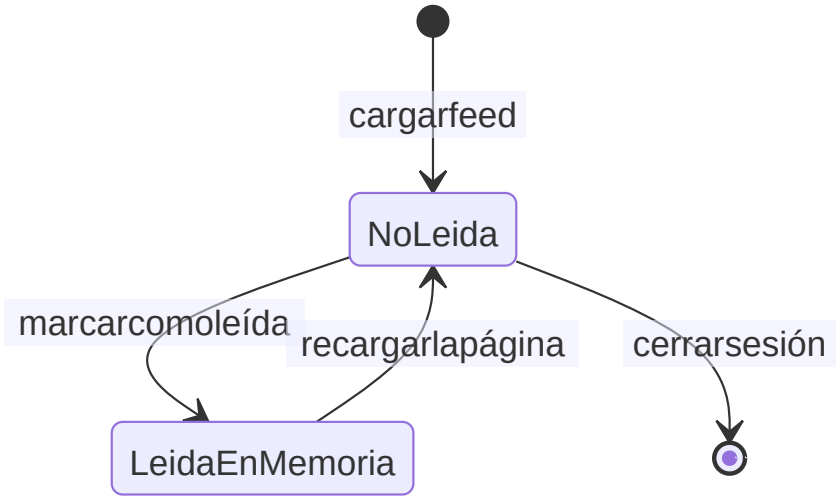



**Figura 20**

*Estados de la campana de notificaciones.*

La campana es provisional: reutiliza las actividades del feed y no persiste el estado leído. Está prevista su conversión en notificaciones persistentes en un ciclo posterior.

## 9. Diagrama ER

El modelo de datos se representa mediante el diagrama entidad-relación de la base de datos MongoDB. Las colecciones principales son usuarios, recetas, interacciones (`likes`, `saves`, `reports`), comentarios, reseñas, seguimientos y actividades; las imágenes residen en GridFS.

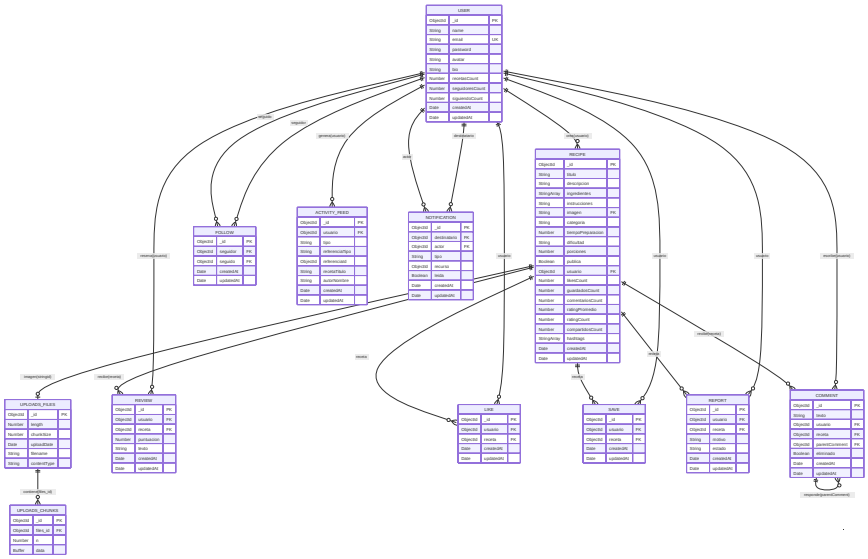


**Figura 21**

*Diagrama entidad-relación del sistema FoodLoop.*

## 10. Requerimientos no funcionales

### 10.1. Arquitectura de tres capas

El desarrollo sigue una arquitectura de tres capas: Frontend, Backend y Base de Datos. En la capa de base de datos se utiliza MongoDB como DBMS, con GridFS para los archivos binarios. En la capa de Backend se usan servicios REST implementados en Node.js con Express y Mongoose, bajo el patrón Modelo–Vista–Controlador, consumidos desde la capa de Frontend. En la capa de Frontend se desarrollan componentes con Angular y TypeScript bajo el patrón de aplicación de página única (SPA), que se comunica con la base de datos a través de los servicios REST del Backend.

### 10.2. Autenticación y autorización

- El registro y el inicio de sesión se realizan con `POST /api/auth/register` y `POST /api/auth/login`.
- Las contraseñas se almacenan con hash bcrypt de 10 rondas; nunca en texto claro.

- La sesión se representa con un JWT de 30 días que viaja en la cabecera `Authorization: Bearer`.
- Todas las rutas, salvo `/api/health`, las de autenticación y la lectura pública de imagen, exigen un token válido; la ausencia o invalidez del token produce `401`.
- La autorización es por propiedad: solo el propietario puede editar, eliminar, cambiar la visibilidad o gestionar la imagen de su receta. Una operación sobre un recurso ajeno o inexistente produce `404`.
- El límite de reintentos es de 10 intentos cada 15 minutos en registro e inicio de sesión.
- **Deuda conocida:** el cierre de sesión no invalida el token en el servidor; el `AuthGuard` del cliente comprueba la presencia del token, pero no su expiración antes de llamar al backend.

### 10.3. Seguridad de datos

- El servidor conserva la propiedad de `_id`, `usuario`, contadores, reportes y métricas: nunca se aceptan desde el cliente.
- Se aplica saneado de entradas (`sanitizeFields`) en comentarios, reseñas, reportes y recetas.
- Se aplican límites de solicitudes: `apiLimiter` global de 100 solicitudes cada 15 minutos, `authLimiter` de 10 cada 15 minutos, `socialLimiter` de 30 por minuto y `createLimiter` de 10 por minuto.
- Los secretos (`MONGODB_URI`, `JWT_SECRET`) deben residir en variables de entorno y no en el repositorio.
- Se distinguen de forma explícita los errores `400`, `401`, `404`, `429` y `500`.
- **Deuda conocida:** no se valida el `ObjectId` en todas las rutas (Mongoose lanza `CastError`); el saneado se aplica ruta a ruta en vez de globalmente; falta `app.set("trust proxy", 1)` para que el límite vea la IP real detrás de Caddy; `uploadLimiter` está definido pero no aplicado a la subida de imágenes; y hay secretos en el historial de `docker-compose.yml` pendientes de rotación.

### 10.4. Almacenamiento

- Los documentos residen en MongoDB en Atlas y las imágenes en el bucket GridFS `uploads`.
- Solo se conserva una imagen vigente por receta; al reemplazar, quitar o borrar la receta se elimina el archivo anterior en GridFS.
- La optimización de imagen se realiza en el navegador con `canvas`: lado mayor de 1600 px y JPEG de calidad 0.8.
- El límite de carga es de 25 MB en el navegador y 5 MB en el servidor.
- La lectura de imagen es pública (`GET /api/recipes/image/:fileId`) y usa `Cache-Control: immutable`.
- **Deuda conocida:** no se verifican los bytes reales del archivo (magic bytes), solo el tipo MIME declarado; borrar una receta no elimina en cascada comentarios, reseñas ni actividad asociada.

### 10.5. Integración

- El cliente Angular consume la API REST bajo el prefijo `/api`, tanto por el proxy local (`proxy.conf.json`) como por Caddy en producción.
- Caddy central actúa como proxy inverso y terminación TLS para `https://foodloopc.duckdns.org`.
- Docker Compose construye y opera los contenedores `recetario-backend` y `recetario-frontend` sobre la red externa `caddy-central_default`.
- El despliegue local usa procesos Node.js y Angular independientes (`npm run dev` y `npm start`).
- Los envíos de archivos usan `multipart/form-data` y `multer-gridfs-storage`.

## 11. Planificación de ciclos de desarrollo

El desarrollo se organiza en ciclos incrementales. La siguiente tabla describe el alcance y las fechas de cada ciclo.

| **Ciclo** | **Qué abarca** | **Fechas** |
|---|---|---|
| Uno | Configuración del entorno, autenticación y CRUD de recetas (P01, P02). | [Completar: fecha] |
| Dos | Imágenes en GridFS y visibilidad pública o privada (P03). | [Completar: fecha] |
| Tres | Red social: interacciones, comentarios, reseñas, seguimiento y feed (P04, P05, P06). | [Completar: fecha] |
| Cuatro | Notificaciones persistentes, perfil público y búsqueda global. | [Completar: fecha] |

## 12. Criterios de aceptación

Los criterios de aceptación definen las condiciones mínimas que debe cumplir el sistema para considerarse aceptado.

### 12.1. Autenticación y gestión de usuarios

- CA-01: Un visitante puede registrarse proporcionando nombre, correo y contraseña. La contraseña debe tener al menos 8 caracteres.
- CA-02: Al registrarse o iniciar sesión con credenciales válidas, el sistema devuelve un token JWT.
- CA-03: Las contraseñas no se almacenan en texto claro; se utilizan hash bcrypt de 10 rondas.
- CA-04: Solo las rutas públicas (/api/health, autenticación y lectura pública de imagen) pueden accederse sin token.
- CA-05: La API responde 401 cuando el token es ausente o inválido.

### 12.2. Recetas e imágenes

- CA-06: El propietario puede crear, editar, eliminar y cambiar la visibilidad de sus recetas.
- CA-07: Solo el propietario puede gestionar la imagen de una receta; al reemplazarla o eliminarla, la imagen anterior se borra de GridFS.
- CA-08: La imagen se sirve públicamente cuando se conoce su identificador.
- CA-09: Las recetas nacen privadas; una receta pública es visible para otros usuarios autenticados.

### 12.3. Interacciones, comentarios y reseñas

- CA-10: Un usuario autenticado puede dar/quitar me gusta, guardar y compartir recetas.
- CA-11: Un usuario autenticado puede comentar, responder de primer nivel y reseñar; una sola reseña por (usuario, receta).
- CA-12: El sistema recalcula el promedio de reseñas cuando se crea, edita o elimina una reseña.
- CA-13: El borrado de comentarios es lógico; las reseñas se gestionan según las reglas de la API.

### 12.4. Seguimiento, feed y notificaciones

- CA-14: Un usuario no puede seguirse a sí mismo; el seguimiento es alternable y único entre par de usuarios.
- CA-15: El feed incluye la actividad propia y la de los usuarios seguidos, con paginación.
- CA-16: La campana muestra la actividad reciente de forma provisional.

### 12.5. Seguridad y calidad

- CA-17: Las entradas se sanean para comentarios, reseñas, reportes y recetas.

- CA-18: Se aplican los límites de solicitud establecidos (api, auth, social, create).
- CA-19: La propiedad de los campos críticos la conserva el servidor.
- CA-20: El sistema distingue correctamente los códigos 400, 401, 404, 429 y 500.

## 13. Firmas de aceptación

Las partes firman en señal de conformidad con el contenido de este documento de especificaciones, requerimientos y criterios de aceptación del sistema FoodLoop.

| **Nombre y puesto** | **Firma y sello** | **Fecha** |
|---|---|---|
| [Completar: contraparte informática] | | |
| [Completar: experto del dominio] | | |
| [Completar: responsable del despliegue] | | |
| [Completar: autor del documento] | | |

*Documento generado a partir de la especificación del proyecto y de la documentación técnica en `docs/`.*

# Anexo A. Catálogo de la interfaz REST

| Método | Ruta | Descripción |
|---|---|---|
| GET | `/api/health` | Estado del servicio y de la conexión con MongoDB. |
| POST | `/api/auth/register` | Registra un usuario y devuelve el token. |
| POST | `/api/auth/login` | Inicia sesión y devuelve el token. |
| POST | `/api/recipes` | Crea una receta y asigna el propietario. |
| GET | `/api/recipes` | Lista las recetas del usuario. |
| GET | `/api/recipes?scope=public` | Lista las recetas públicas y las propias. |
| GET | `/api/recipes/:id` | Detalle de una receta propia o pública. |
| PUT | `/api/recipes/:id` | Actualiza una receta propia. |
| DELETE | `/api/recipes/:id` | Elimina una receta propia y su imagen. |
| GET | `/api/recipes/image/:fileId` | Sirve la imagen de una receta. |
| POST | `/api/recipes/:id/image` | Sube o reemplaza la imagen de una receta. |
| POST | `/api/recipes/:id/like` | Alterna el me gusta. |
| POST | `/api/recipes/:id/save` | Alterna el guardado. |
| POST | `/api/recipes/:id/share` | Incrementa el contador de compartidos. |
| POST | `/api/recipes/:id/report` | Reporta una receta con un motivo. |
| POST | `/api/comments` | Crea un comentario o respuesta. |
| GET | `/api/comments/receta/:recetaId` | Lista los comentarios de una receta. |
| PUT | `/api/comments/:id` | Edita un comentario propio. |
| DELETE | `/api/comments/:id` | Retira un comentario propio. |
| POST | `/api/reviews` | Crea una reseña. |
| GET | `/api/reviews/receta/:recetaId` | Lista las reseñas de una receta. |
| PUT | `/api/reviews/:id` | Actualiza una reseña propia. |
| DELETE | `/api/reviews/:id` | Elimina una reseña propia. |
| POST | `/api/follows` | Sigue o deja de seguir a un usuario. |
| GET | `/api/follows/followers/:userId` | Lista los seguidores de un usuario. |

| **Método** | **Ruta** | **Descripción** |
|---|---|---|
| GET | `/api/follows/following/:userId` | Lista los usuarios seguidos. |
| GET | `/api/follows/check/:userId` | Indica si se sigue a un usuario. |
| GET | `/api/activity/feed` | Feed paginado de actividad. |
| GET | `/api/activity/user/:userId` | Actividad de un usuario. |

## Anexo B. Estados y sus transiciones

| **Entidad** | **Estados y transiciones permitidas** |
|---|---|
| Receta | Nace *privada*; puede alternar a *pública* y de nuevo a *privada*. El atributo *reportada* es independiente y no cambia la visibilidad. La entidad se elimina de forma física. |
| Imagen | *Sin imagen* → *con imagen* (al subir); *con imagen* → *con imagen nueva* (al reemplazar); *con imagen* → *sin imagen* (al quitar o borrar la receta). Solo una imagen vigente por receta. |
| Comentario | *Visible* → *eliminado* de forma lógica; la respuesta de primer nivel comparte el ciclo del comentario principal. |
| Reseña | *Creada* → *actualizada* (por el mismo autor) → *eliminada*; una sola reseña por usuario y receta. |
| Seguimiento | *No sigue* → *sigue* → *no sigue*; pareja única entre emisor y destinatario, sin auto-seguimiento. |
| Sesión | *Activa* con token válido → *cerrada* localmente al eliminar el token; el token permanece válido en el servidor hasta su expiración. |